

ARCHITECTURAL NOTE

Research Node / Backward Node Concept v0.1

Structured preservation of unresolved architectural futures under bounded constraints.

Author: Kotov Ivan

Place: Bruxelles

Year: 2026

DOCUMENT FRAME

An architectural note on how unresolved failures are preserved as structured research objects instead of disappearing into intuition, chat residue, or vague future work.

AUTHOR

Kotov Ivan

PLACE

Bruxelles

DATE

2026

READING NOTE. This styled edition preserves the original structure while presenting the note in a publication-ready format suitable for review, sharing, and archival use.

1. Purpose

`Research Node / Backward Node` defines what happens **after** a path has already encountered an `L4 Glitch`.

Its purpose is not to repair reality by rhetoric. Its purpose is to preserve the break as a **structured research object**.

A broken branch should not disappear into intuition, chat residue, or vague “future work.” It should become a bounded node that records:

- where the path failed,
- which constraint actually broke,
- what speculative bridge was imagined,
- what evidence would be required to reopen the path,
- and under what authority the node may later be revisited.

In short:

`GlitchNode` marks the failure. `ResearchNode` preserves the unresolved future opened by that failure.

2. Core distinction

A `Research Node` is **not**:

- a patch,
- an override,
- a permission grant,
- a hidden wish-list,
- or a fantasy branch silently smuggled into execution.

A `Research Node` is a **disciplined placeholder for missing reality**.

It says:

this path is not currently valid, but the invalidity itself has informational value, and the missing bridge can be described precisely enough to guide future inquiry.

That distinction matters.

Without it, speculative architecture collapses into either:

- sterile refusal,
- or unbounded imagination.

Research Node is the middle object that preserves ambition **without granting false legitimacy**.

3. Terminology

3.1 GlitchNode

A formal marker that an execution, scenario, or architectural branch has failed under L4 or another hard constraint.

3.2 ResearchNode

A structured node created **because** a **GlitchNode** occurred. It stores the unresolved bridge as a bounded research target.

3.3 BackwardNode

A specific interpretation of **ResearchNode** in which the future path is imagined first, and the node points backward toward the missing condition that would make that future possible.

Example:

- desired future branch: low-energy autonomous ocean maintenance swarm;
- current dead end: no feasible materials + repair + energy chain;
- backward node: “this branch would only be reopenable if material/process X reached constraint profile Y.”

So:

- **ResearchNode** is the general object;
- **BackwardNode** is the directional logic: **future target -> missing bridge -> research demand**.

4. Why this layer is needed

Most systems handle failure badly in one of two ways:

4.1 Failure disappears

The branch fails, and the failure is lost inside logs, conversations, comments, or memory fragments.

Result:

- no structured learning,
- no provenance,
- no re-entry point,
- no auditability.

4.2 Failure is overridden by optimism

The branch fails, but someone informally says “assume better batteries,” “assume a smarter model,” “assume quantum acceleration,” “assume better material science.”

Result:

- fantasy is silently promoted into architecture,
- provenance is destroyed,
- authority is faked,
- and downstream readers cannot tell what is real versus assumed.

`Research Node` exists to stop both failures.

5. Minimal lifecycle

Stage 1 — Branch execution or scenario traversal

A path is explored.

Stage 2 — Constraint failure

A `GlitchNode` is emitted. The branch is marked invalid under current conditions.

Stage 3 — Node extraction

A `ResearchNode` is created from the failure. The broken path is converted into a structured research object.

Stage 4 — Quarantine

The branch remains non-executable. The `ResearchNode` does **not** reopen the path by itself.

Stage 5 — Evidence accumulation

Research, simulation, external validation, or real-world testing may add evidence to the node.

Stage 6 — Re-evaluation

Only after sufficient evidence exists may the path be reviewed for transition from:

- dead end,
- to plausible research,
- to conditionally grounded,
- to validated under explicit scope.

6. Object model

Below is a minimal object structure.

ResearchNode

- research_node_id
- source_glitch_id
- parent_path_id
- node_mode # research | backward
- failure_scope # local | cross-layer | systemic
- failed_constraint_type # energy | time | trust | privilege | embodiment | evidence | maintenance | etc.
- failed_constraint_detail
- dead_end_description
- imagined_bridge_summary
- bridge_class # material | algorithmic | governance | embodiment | supply-chain | scientific-unknown
- required_evidence_type # experiment | simulation | formal proof | field validation | compliance evidence
- evidence_threshold
- current_status # quarantined | exploratory | under-review | reopenable | rejected
- authority_scope # who may edit / review / reopen
- witness_requirement
- uncertainty_level
- reversibility_assessment
- downstream_risk_if_wrong
- created_by_c
- human_anchor_a
- timestamp_created
- timestamp_last_reviewed
- linked_artifacts[] # DEA / EA / witness events / papers / simulations / code refs
- notes

This is intentionally stricter than a note or ticket. A `ResearchNode` is not a sticky note for hope. It is a typed object.

7. Status model

A `ResearchNode` should not drift ambiguously. A minimal status model:

7.1 `quarantined`

The default state. The branch is broken. The node exists only as a preserved research target.

7.2 `exploratory`

Some investigation has begun. No reopening authority is implied.

7.3 under-review

Evidence exists and is being evaluated. Still not executable.

7.4 reopenable

The node has accumulated enough support for bounded re-testing under explicit constraints. Not equivalent to validation.

7.5 rejected

The imagined bridge has been judged non-viable, internally incoherent, or no longer worth pursuing. The node remains preserved for lineage, but the path stays closed.

8. Relationship to authority

This point is critical.

A ResearchNode may preserve value. It does **not** grant legitimacy.

That means:

- learning does not imply authority,
- speculation does not imply permission,
- interest does not imply admissibility,
- and preserved futures do not become executable merely because they are attractive.

A node can be brilliant and still remain quarantined.

This keeps the architecture clean.

Otherwise, every speculative branch becomes a political pressure vector against reality.

9. Relationship to $c = a + b$

In a $c = a + b$ architecture, a ResearchNode does not belong to the model alone. It belongs to the continuity-bearing system c, under a human anchor a and procedural substrate b.

That matters because a node is not just a memory item. It is a **continuity-bearing unresolved obligation**.

The system remembers:

- what broke,
- why it broke,
- what future was imagined,
- and what must be proven before the future stops being fiction.

So a `ResearchNode` is a way for `c` to carry unresolved reality **without lying about it**.

10. Relationship to L4

L4 remains the boundary.

A `ResearchNode` does not bypass L4. It exists because L4 held.

In other words:

- `GlitchNode` says: reality refused this path;
- `ResearchNode` says: the refusal itself is now a structured object.

Typical L4-driven triggers:

- energy cost exceeds feasible envelope,
- time windows collapse the path,
- maintenance or repair burden breaks persistence,
- privilege requirements exceed safe delegation,
- evidence chain is too weak,
- embodiment assumptions are ungrounded,
- trust surfaces become non-auditable,
- continuity would fragment under replacement or escalation.

The node exists so those breaks remain visible and actionable for inquiry.

11. Relationship to DEA / EA-L4 / EATP / Witness

11.1 DEA

`DEA` helps define when inert input becomes experience. A `ResearchNode` may be created from a failed interpretation path when an input does not merely get stored, but reveals a persistent unresolved constraint.

11.2 EA-L4 / EATP

If a failed branch survives long enough to become a bounded, consequence-bearing record, parts of it may later contribute to `Experience Artifacts`. But the node itself must not be confused with validated EA. It begins as quarantined unresolved structure.

11.3 L4 Witness

Every important mutation of a `ResearchNode` should be witnessable:

- creation,
- status transition,
- evidence attachment,
- review outcome,
- rejection or bounded reopening.

This prevents speculative bridges from being rewritten silently after the fact.

12. Example

Scenario

A future branch proposes an autonomous household robot that may coordinate heat, food logistics, emergency response, and private memory continuity inside the home.

Failure

The branch hits an L4 lock:

- too much privileged action concentrated in one body,
- unreliable local fallback during network/power loss,
- insufficient witnessability for boundary-crossing actions.

GlitchNode

The system emits:

- `lock_type = privilege_lock`
- `failed_constraint = unbounded household authority concentration`

ResearchNode

A node is created:

- `imagined_bridge_summary = distributed role decomposition with explicit witness trail and degraded fallback modes`
- `required_evidence_type = simulation + field validation + safety review`
- `status = quarantined`

Now the future branch is not lost. But it is also not allowed to masquerade as ready architecture.

13. Hidden and explicit bridges

Explicit bridge

`Glitch -> ResearchNode -> bounded future inquiry`

This is the direct bridge between failure and structured continuation.

Hidden bridge 1

Cybernetics / Ashby: If the regulator cannot preserve distinctions between dead ends, speculative bridges, and validated paths, the control system loses variety and collapses into confusion.

Hidden bridge 2

Information theory / provenance: A quarantined node preserves signal about missing reality. Silent patching destroys that signal by replacing uncertainty with narrative compression.

Hidden bridge 3

Biology / wound logic: A scar is not a failure erased. It is tissue memory that changes future behavior. `ResearchNode` plays a similar role for architectural cognition.

14. Earth paragraph

In engineering, when a prototype fails under load, the broken part is not thrown away together with the lesson. The fracture location, the material behavior, the heat profile, and the timing of failure become part of the next design cycle. A broken turbine blade,

a cracked beam, or a burnt connector is not “fantasy.” It is evidence that a future design must answer. A `ResearchNode` is the architectural equivalent of preserving the fracture, not hiding it.

15. A/B slots for safe future evolution

A-slot — strict mode

A `ResearchNode` is only a quarantined, typed, witness-backed object for preserving unresolved futures. No implied authority. No speculative reopening without explicit evidence.

B-slot — exploratory mode

Future work may allow richer semantics:

- ranked bridge classes,
- multi-node dependency graphs,
- research portfolio prioritization,
- shared inter-entity research coordination,
- cinematic walkthroughs.

If source discipline weakens, revert to A-slot.

16. Final line

A system becomes dangerous when it cannot distinguish between:

- what failed,
- what is imagined,
- what is being researched,
- and what is actually allowed to become real.

`Research Node / Backward Node` is the object that keeps those categories apart.

It does not close the gap. It preserves the gap honestly enough that future work can meet it.